

Game.js:

```
// =====  
// PACMAN GAME - game.js  
// =====  
  
// ---- Canvas ----  
const canvas = document.getElementById("gameCanvas");  
const ctx = canvas.getContext("2d");  
  
// Logical tile size (we scale visually with CSS if needed)  
const TILE_SIZE = 24;  
  
// ---- Classic-style Maze ----  
// # = wall, . = pellet, o = power pellet, ' ' = empty path  
const MAZE_TEMPLATE = [  
  "#####",  
  "#.....##.....#",  
  "# #####.##.#####.#",  
  "#O####.#####.####O#",  
  "# #####.##.#####.##.#",  
  "#.....#",  
  "# #####.#####.#####.#",  
  "# #####.#####.#####.#",  
  "#.....##.....#",  
  "#####.#####.#####",  
  "# #####.#####.#####.#",  
  "#.....#", // ghosts start on this row  
  "# #####.#####.#####.#",  
  "# #####.#####.#####.#",  
  "#O.##.....##.O#",  
  "###.##.#####.##.###",  
  "#.....##.....#",  
  "# #####.#####.#####.#",  
  "#.....#", // pacman start row  
  "#####"  
];
```

```
const ROWS = MAZE_TEMPLATE.length;
const COLS = MAZE_TEMPLATE[0].length;

canvas.width = COLS * TILE_SIZE;
canvas.height = ROWS * TILE_SIZE;

// ----- Maze grid (mutable pellets) -----
const maze = MAZE_TEMPLATE.map(row => row.split(''));

// Tile helpers
function isWall(row, col) {
  if (row < 0 || row >= ROWS || col < 0 || col >= COLS) return true;
  return maze[row][col] === "#";
}

function isWalkable(row, col) {
  if (row < 0 || row >= ROWS || col < 0 || col >= COLS) return false;
  return maze[row][col] !== "#";
}

function tileAt(x, y) {
  return {
    row: Math.floor(y),
    col: Math.floor(x)
  };
}

// ----- Assets -----
function loadImage(path) {
  const img = new Image();
  img.src = path;
  return img;
}

// Pellets
const pelletImg = loadImage("img/pellet.png");
```

```

const powerPelletImg = loadImage("img/power_pellet.png");

// Pac-Man sprites
const pacmanSprites = {
  right: [1, 2, 3].map(i => loadImage(`img/pacman/pacman-right/${i}.png`)),
  left: [1, 2, 3].map(i => loadImage(`img/pacman/pacman-left/${i}.png`)),
  up: [1, 2, 3].map(i => loadImage(`img/pacman/pacman-up/${i}.png`)),
  down: [1, 2, 3].map(i => loadImage(`img/pacman/pacman-down/${i}.png`))
};

// Ghost sprites (note: filename "frightened.png" to match your folder)
const ghostSprites = {
  red: loadImage("img/ghosts/red.png"),
  pink: loadImage("img/ghosts/pink.png"),
  teal: loadImage("img/ghosts/teal.png"),
  orange: loadImage("img/ghosts/orange.png"),
  frightened: loadImage("img/ghosts/frightened.png")
};

// Sounds
const sndBegin = new Audio("sounds/pacman_beginning.wav");
const sndChomp = new Audio("sounds/pacman_chomp.wav");
const sndDeath = new Audio("sounds/pacman_death.wav");
const sndEatFruit = new Audio("sounds/pacman_eatfruit.wav");
const sndEatGhost = new Audio("sounds/pacman_eatghost.wav");
const sndIntermission = new Audio("sounds/pacman_intermission.wav");

sndChomp.loop = true;
let chompPlaying = false;

// ----- Meme popup (safe even if empty) -----
const memeFiles = [
  // e.g. "memes/meme1.png", "memes/meme2.jpg"
  // Leave empty while testing; code will just skip.
];

let memeImage = null;
let memeTimer = 0; // seconds remaining

```

```
function triggerMeme() {  
  if (memeFiles.length === 0) return; // nothing to show  
  const path = memeFiles[Math.floor(Math.random() * memeFiles.length)];  
  memedImage = loadImage(path);  
  memeTimer = 3; // show for 3 seconds  
}
```

```
// ----- Game State -----
```

```
let score = 0;  
let lives = 3;  
let gameState = "playing"; // "playing", "dead", "gameover", "win"
```

```
const PACMAN_SPEED = 7; // tiles per second  
const GHOST_SPEED = 6;  
const POWER_DURATION = 7; // seconds
```

```
// Pac-Man object
```

```
const pacman = {  
  x: 13.5, // center column  
  y: 18.5, // row 18 (second from bottom, path row)  
  dirX: 0,  
  dirY: 0,  
  nextDirX: 0,  
  nextDirY: 0,  
  speed: PACMAN_SPEED,  
  facing: "right",  
  animFrame: 0,  
  animTimer: 0,  
  power: false,  
  powerTimer: 0  
};
```

```
// Helper: create ghost
```

```
function makeGhost(type, x, y) {  
  return {  
    type,  
    x,  
    y,  
    dirX: 0,  
    dirY: 0,  
    nextDirX: 0,  
    nextDirY: 0,  
    speed: GHOST_SPEED,  
    facing: "right",  
    animFrame: 0,  
    animTimer: 0,  
    power: false,  
    powerTimer: 0  
  };  
}
```

```

    x,
    y,
    dirX: 0,
    dirY: 0,
    speed: GHOST_SPEED,
    frightened: false,
    eaten: false,
    released: false, // start locked
    releaseTimer: 0 // seconds until they start moving
  };
}

// Ghosts start in the center row (row 11) far above Pac-Man
const ghosts = [
  makeGhost("red", 13.5, 11.5),
  makeGhost("pink", 14.5, 11.5),
  makeGhost("teal", 12.5, 11.5),
  makeGhost("orange", 15.5, 11.5)
];

// ghost release delays in SECONDS
const ghostReleaseDelays = [0, 2, 4, 6]; // seconds: Blinky, Pinky, Inky, Clyde

ghosts.forEach((g, i) => {
  g.releaseTimer = ghostReleaseDelays[i];
  g.released = (ghostReleaseDelays[i] === 0); // first ghost moves immediately
});

// ----- Input -----
window.addEventListener("keydown", e => {
  let dx = 0, dy = 0;
  if (e.key === "ArrowUp") { dx = 0; dy = -1; pacman.facing = "up"; }
  if (e.key === "ArrowDown") { dx = 0; dy = 1; pacman.facing = "down"; }
  if (e.key === "ArrowLeft") { dx = -1; dy = 0; pacman.facing = "left"; }
  if (e.key === "ArrowRight") { dx = 1; dy = 0; pacman.facing = "right"; }

  if (dx !== 0 || dy !== 0) {

```

```

    pacman.nextDirX = dx;
    pacman.nextDirY = dy;
}

if (gameState === "dead" && e.key === " ") {
    resetAfterDeath();
} else if ((gameState === "gameover" || gameState === "win") &&
    (e.key === "r" || e.key === "R")) {
    restartGame();
}
});

// ----- Movement helpers -----
function atTileCenter(entity) {
    // Tile centers are at (floor + 0.5, floor + 0.5)
    const cx = Math.floor(entity.x) + 0.5;
    const cy = Math.floor(entity.y) + 0.5;
    return Math.abs(entity.x - cx) < 0.1 && Math.abs(entity.y - cy) < 0.1;
}

function tryChangeDirection(entity, nextX, nextY, dt) {
    if (nextX === 0 && nextY === 0) return;
    if (!atTileCenter(entity)) return;

    // Use the tile we are actually standing in
    const centerRow = Math.floor(entity.y);
    const centerCol = Math.floor(entity.x);
    const targetRow = centerRow + nextY;
    const targetCol = centerCol + nextX;

    if (!isWall(targetRow, targetCol)) {
        entity.dirX = nextX;
        entity.dirY = nextY;
    }
}

function moveEntity(entity, dt) {

```

```

const step = entity.speed * dt; // tiles per frame
let newX = entity.x + entity.dirX * step;
let newY = entity.y + entity.dirY * step;

// Tunnel wrap horizontally
if (newX < 0)    newX = COLS - 0.01;
if (newX > COLS-1) newX = 0.01;

const { row, col } = tileAt(newX, newY);
if (!isWall(row, col)) {
    entity.x = newX;
    entity.y = newY;
} else {
    // Hit wall -> snap exactly to tile center (floor + 0.5)
    entity.x = Math.floor(entity.x) + 0.5;
    entity.y = Math.floor(entity.y) + 0.5;
    entity.dirX = 0;
    entity.dirY = 0;
}
}

// ----- Game logic: Pac-Man -----
function updatePacman(dt) {
    if (gameState !== "playing") return;

    // Direction change at intersections
    tryChangeDirection(pacman, pacman.nextDirX, pacman.nextDirY, dt);

    // Movement
    moveEntity(pacman, dt);

    // Animation
    if (pacman.dirX !== 0 || pacman.dirY !== 0) {
        pacman.animTimer += dt;
        if (pacman.animTimer > 0.08) {
            pacman.animTimer = 0;
            pacman.animFrame = (pacman.animFrame + 1) % 3;
        }
    }
}

```

```

} else {
  pacman.animFrame = 1; // mouth half-open idle
}

// Pellets / power pellets
const { row, col } = tileAt(pacman.x, pacman.y);
const tile = maze[row][col];

if (tile === "." || tile === "o") {
  maze[row][col] = " ";

  if (tile === ".") {
    score += 10;
    // Start chomp loop while eating
    if (!chompPlaying) {
      sndChomp.currentTime = 0;
      sndChomp.play().catch(()=>{});
      chompPlaying = true;
    }
  } else if (tile === "o") {
    score += 50;
    pacman.power = true;
    pacman.powerTimer = POWER_DURATION;
    ghosts.forEach(g => { g.frightened = true; g.eaten = false; });
    sndEatFruit.currentTime = 0;
    sndEatFruit.play().catch(()=>{});
    triggerMeme(); // meme popup on power pellet
  }

  checkWin();
}

// Stop chomp sound when not on pellet
if (tile !== "." && chompPlaying) {
  sndChomp.pause();
  sndChomp.currentTime = 0;
  chompPlaying = false;
}

```



```

}

// Power mode timer
if (pacman.power) {
  pacman.powerTimer -= dt;
  if (pacman.powerTimer <= 0) {
    pacman.power = false;
    ghosts.forEach(g => { g.frightened = false; g.eaten = false; });
  }
}
}

```

// ----- Game logic: Ghosts -----

```

function chooseGhostDirection(ghost) {
  if (!atTileCenter(ghost)) return;

  const centerRow = Math.floor(ghost.y);
  const centerCol = Math.floor(ghost.x);

  const dirs = [
    { x: 1, y: 0 },
    { x: -1, y: 0 },
    { x: 0, y: 1 },
    { x: 0, y: -1 }
  ];

  const opp = { x: -ghost.dirX, y: -ghost.dirY };
  const options = [];

  dirs.forEach(d => {
    const r = centerRow + d.y;
    const c = centerCol + d.x;
    if (!isWall(r,c)) {
      // don't immediately reverse unless forced
      if (!(d.x === opp.x && d.y === opp.y)) {
        options.push(d);
      }
    }
  });
}

```

```

    }
  });

  if (options.length === 0) {
    // must reverse
    ghost.dirX = opp.x;
    ghost.dirY = opp.y;
    return;
  }

  // Simple "chase-ish": prefer directions that reduce distance to Pac-Man in chase,
  // random when frightened.
  let chosen;
  if (ghost.frightened) {
    chosen = options[Math.floor(Math.random() * options.length)];
  } else {
    let best = null;
    let bestDist = Infinity;
    for (const d of options) {
      const nx = ghost.x + d.x;
      const ny = ghost.y + d.y;
      const dx = nx - pacman.x;
      const dy = ny - pacman.y;
      const dist = dx*dx + dy*dy;
      if (dist < bestDist) {
        bestDist = dist;
        best = d;
      }
    }
    chosen = best;
  }

  ghost.dirX = chosen.x;
  ghost.dirY = chosen.y;
}

function updateGhosts(dt) {

```

```

if (gameState !== "playing") return;

ghosts.forEach(g => {
  // 1) Handle staggered release based on releaseTimer
  if (!g.released) {
    g.releaseTimer -= dt; // dt is in SECONDS
    if (g.releaseTimer <= 0) {
      g.released = true;
      // give an initial direction so they start moving right away
      g.dirX = 0;
      g.dirY = -1; // start by going up
    } else {
      // still waiting in place this frame
      return;
    }
  }
}

// 2) Normal movement / frightened slowdown
const baseSpeed = g.frightened ? GHOST_SPEED * 0.7 : GHOST_SPEED;
g.speed = baseSpeed;

chooseGhostDirection(g);
moveEntity(g, dt);

// 3) Collision with Pac-Man
const dx = g.x - pacman.x;
const dy = g.y - pacman.y;
const dist2 = dx*dx + dy*dy;

if (dist2 < 0.35*0.35) {
  if (g.frightened && !g.eaten) {
    // Ghost eaten
    g.eaten = true;
    g.frightened = false;
    score += 200;
    sndEatGhost.currentTime = 0;
    sndEatGhost.play().catch(()=>{});
  }
}

```

```

    // send ghost back to start row and lock again for a few seconds
    g.x = 13.5;
    g.y = 11.5;
    g.dirX = 0;
    g.dirY = 0;
    g.releaseTimer = 4; // wait 4 seconds before rejoining
    g.released = false;
  } else if (!g.frightened) {
    // Pac-Man dies
    handleDeath();
  }
}
});
}

```

```

function handleDeath() {
  if (gameState !== "playing") return;
  lives--;
  sndDeath.currentTime = 0;
  sndDeath.play().catch(()=>{});
  sndChomp.pause();
  sndChomp.currentTime = 0;
  chompPlaying = false;

  if (lives <= 0) {
    gameState = "gameover";
  } else {
    gameState = "dead";
  }
}

```

```

function resetAfterDeath() {
  // reset pacman
  pacman.x = 13.5;
  pacman.y = 18.5;
  pacman.dirX = 0;
  pacman.dirY = 0;

```

```

pacman.nextDirX = 0;
pacman.nextDirY = 0;
pacman.power = false;
pacman.powerTimer = 0;

// reset ghosts to starting row
const positions = [
  { x: 13.5, y: 11.5 },
  { x: 14.5, y: 11.5 },
  { x: 12.5, y: 11.5 },
  { x: 15.5, y: 11.5 }
];
ghosts.forEach((g,i) => {
  g.x = positions[i].x;
  g.y = positions[i].y;
  g.dirX = 0;
  g.dirY = 0;
  g.frightened = false;
  g.eaten = false;
  g.releaseTimer = ghostReleaseDelays[i];
  g.released = (ghostReleaseDelays[i] === 0);
});

gameState = "playing";
}

function restartGame() {
  // reset maze pellets
  for (let r = 0; r < ROWS; r++) {
    for (let c = 0; c < COLS; c++) {
      const ch = MAZE_TEMPLATE[r][c];
      if (ch === "." || ch === "o" || ch === "#" || ch === " ") {
        maze[r][c] = ch;
      } else {
        maze[r][c] = " ";
      }
    }
  }
}

```

```

    }

    score = 0;
    lives = 3;
    pacman.power = false;
    pacman.powerTimer = 0;
    resetAfterDeath();
    gameState = "playing";
}

function checkWin() {
    for (let r = 0; r < ROWS; r++) {
        for (let c = 0; c < COLS; c++) {
            if (maze[r][c] === "." || maze[r][c] === "o") {
                return;
            }
        }
    }
    gameState = "win";
}

// ----- Drawing -----
function drawMaze() {
    ctx.fillStyle = "black";
    ctx.fillRect(0,0,canvas.width,canvas.height);

    for (let r = 0; r < ROWS; r++) {
        for (let c = 0; c < COLS; c++) {
            const chTemplate = MAZE_TEMPLATE[r][c];
            const tileCurrent = maze[r][c];
            const x = c * TILE_SIZE;
            const y = r * TILE_SIZE;

            // Walls from template
            if (chTemplate === "#") {
                ctx.fillStyle = "#001a66";
                ctx.fillRect(x, y, TILE_SIZE, TILE_SIZE);
                ctx.strokeStyle = "#00bfff";
            }
        }
    }
}

```

```

    ctx.lineWidth = 4;
    ctx.strokeRect(x+2, y+2, TILE_SIZE-4, TILE_SIZE-4);
}

// Pellets from current maze (so they disappear)
if (tileCurrent === ".") {
    const cx = x + TILE_SIZE/2;
    const cy = y + TILE_SIZE/2;
    if (pelletImg.complete && pelletImg.naturalWidth) {
        const size = TILE_SIZE * 0.3;
        ctx.drawImage(pelletImg, cx-size/2, cy-size/2, size, size);
    } else {
        ctx.fillStyle = "#eee";
        ctx.beginPath();
        ctx.arc(cx, cy, 2, 0, Math.PI*2);
        ctx.fill();
    }
} else if (tileCurrent === "o") {
    const cx = x + TILE_SIZE/2;
    const cy = y + TILE_SIZE/2;
    const size = TILE_SIZE * 0.8;
    if (powerPelletImg.complete && powerPelletImg.naturalWidth) {
        ctx.drawImage(powerPelletImg, cx-size/2, cy-size/2, size, size);
    } else {
        ctx.fillStyle = "#fff";
        ctx.beginPath();
        ctx.arc(cx, cy, TILE_SIZE*0.3, 0, Math.PI*2);
        ctx.fill();
    }
}
}
}
}

function drawPacman() {
    const px = pacman.x * TILE_SIZE;
    const py = pacman.y * TILE_SIZE;

```

```

const frames = pacmanSprites[pacman.facing] || pacmanSprites.right;
const img = frames[pacman.animFrame];

const size = TILE_SIZE * 0.9;
if (img.complete && img.naturalWidth) {
  ctx.drawImage(img, px - size/2, py - size/2, size, size);
} else {
  ctx.fillStyle = "yellow";
  ctx.beginPath();
  ctx.arc(px, py, TILE_SIZE*0.4, 0, Math.PI*2);
  ctx.fill();
}
}

function drawGhosts() {
  ghosts.forEach(g => {
    const gx = g.x * TILE_SIZE;
    const gy = g.y * TILE_SIZE;
    let img;
    if (g.frightened && !g.eaten) {
      img = ghostSprites.frightened;
    } else {
      img = ghostSprites[g.type];
    }

    const size = TILE_SIZE * 0.9;
    if (img && img.complete && img.naturalWidth) {
      ctx.drawImage(img, gx - size/2, gy - size/2, size, size);
    } else {
      ctx.fillStyle = g.frightened ? "#0000ff" : "#ff0000";
      ctx.beginPath();
      ctx.arc(gx, gy, TILE_SIZE*0.4, Math.PI, 0);
      ctx.lineTo(gx+TILE_SIZE*0.4, gy+TILE_SIZE*0.4);
      ctx.lineTo(gx-TILE_SIZE*0.4, gy+TILE_SIZE*0.4);
      ctx.closePath();
      ctx.fill();
    }
  })
}

```



```

});
}

function drawHUD() {
  ctx.fillStyle = "#00ffff";
  ctx.font = "24px Arial";
  ctx.textAlign = "left";
  ctx.textBaseline = "top";
  ctx.fillText("Score: " + score, 10, 5);

  const livesText = "Lives: " + lives;
  const w = ctx.measureText(livesText).width;
  ctx.textAlign = "right";
  ctx.fillText(livesText, canvas.width - 10, 5);

  if (gameState === "gameover" || gameState === "win") {
    ctx.fillStyle = "rgba(0,0,0,0.6)";
    ctx.fillRect(0, canvas.height/2 - 40, canvas.width, 80);
    ctx.fillStyle = "#fff";
    ctx.font = "32px Arial";
    ctx.textAlign = "center";
    const text = gameState === "gameover" ? "GAME OVER - Press R" : "YOU WIN! - Press R";
    ctx.fillText(text, canvas.width/2, canvas.height/2);
  } else if (gameState === "dead") {
    ctx.fillStyle = "rgba(0,0,0,0.6)";
    ctx.fillRect(0, canvas.height/2 - 40, canvas.width, 80);
    ctx.fillStyle = "#fff";
    ctx.font = "32px Arial";
    ctx.textAlign = "center";
    ctx.fillText("You Died - Press Space", canvas.width/2, canvas.height/2);
  }
}

function drawMemeOverlay() {
  if (memeTimer <= 0 || !memeImage || !memeImage.complete || !memeImage.naturalWidth) return;

  ctx.fillStyle = "rgba(0,0,0,0.7)";

```

```

ctx.fillRect(0,0,canvas.width,canvas.height);

const maxW = canvas.width * 0.6;
const maxH = canvas.height * 0.6;
let w = memelImage.width;
let h = memelImage.height;
const s = Math.min(maxW / w, maxH / h, 1);
w *= s; h *= s;
const x = (canvas.width - w)/2;
const y = (canvas.height - h)/2;
ctx.drawImage(memelImage, x, y, w, h);
}

// ----- Main loop -----
let lastTime = performance.now();

function gameLoop(timestamp) {
  const dt = (timestamp - lastTime) / 1000; // seconds
  lastTime = timestamp;

  if (gameState === "playing") {
    updatePacman(dt);
    updateGhosts(dt);
  }

  if (memeTimer > 0) {
    memeTimer -= dt;
    if (memeTimer < 0) memeTimer = 0;
  }

  drawMaze();
  drawPacman();
  drawGhosts();
  drawHUD();
  if (gameState === "playing") {
    drawMemeOverlay();
  }
}

```

```
    requestAnimationFrame(gameLoop);
  }

  // Start the game
  restartGame();
  requestAnimationFrame(gameLoop);
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pac-Man Neon</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <canvas id="gameCanvas" width="840" height="600"></canvas>

  <!-- Meme overlay -->
  <img id="memeOverlay">

  <script src="game.js"></script>
</body>
</html>
```

Index.css

```
/* ===== Global reset & layout ===== */

html, body {
  margin: 0;
  padding: 0;
  height: 100%;
```

```
}

/* Center everything on the page */
body {
  display: flex;
  justify-content: center; /* horizontal centering */
  align-items: center;     /* vertical centering */
  background: #000000;     /* black background like original Pac-Man */
  font-family: Arial, sans-serif;
  color: #ffffff;
}

/* ===== Game canvas ===== */

#gameCanvas {
  display: block;

  /* Let the game scale nicely on bigger screens */
  width: 90vmin;    /* 90% of the smaller viewport dimension */
  height: auto;     /* keep aspect ratio based on the JS-set size */

  /* Optional: add a subtle glow border around the game */
  box-shadow: 0 0 20px rgba(0, 191, 255, 0.6);
  border-radius: 8px;
}

/* If you ever wrap the game in a container, this will
   keep it centered too (safe to leave even if unused). */

#game-wrapper {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```